

ADVENTURE-2600: A Protocol for Camera-Mediated Agent Evaluation on an Original Atari 2600

Anonymous authors

[TODO: replace with author names, affiliations, and contact]

[TODO: replace with submission date]

Abstract

Atari benchmarks have played an influential role in reinforcement-learning research, but most reported evaluation is performed through software environments with access to convenient resets, frame buffers, and score signals. We present a preregistered experimental protocol for evaluating an agent on an unmodified original Atari 2600. The agent is trained in an emulator and then plays *Adventure* by observing a physical game display through a fixed USB camera and by issuing only standard joystick switch closures through an external controller bridge. The system records synchronized video, requested actions, applied relay states, and timing so that behaviour can be inspected and replayed. This manuscript defines the hardware boundary, transfer procedure, baselines, metrics, and reporting requirements; it reports no empirical performance results. It neither seeks to reproduce Agent57’s Atari57 result nor to claim general or human-level intelligence. The objective is to make a future real-hardware result falsifiable and reproducible.

1 Introduction

The Arcade Learning Environment (ALE) made Atari 2600 games a common platform for studying general-purpose reinforcement-learning algorithms [2]. Deep Q-Networks and their descendants established strong Atari results [4, 3], and Agent57 emphasized breadth across the 57-game suite [1]. For a recent review of DeepMind’s game-RL trajectory—including DQN-style, model-free approaches and the model-based progression through AlphaGo, AlphaGo Zero, and MuZero—see Shaheen et al. [5]. These software benchmarks are valuable, but their interfaces are not the same as an agent operating a physical console: a deployed system must cope with video capture, display/camera distortion, latency, external actuation, and less convenient episode control.

This paper scopes a narrow real-hardware experiment. We ask whether an agent trained in an Atari emulator can transfer to *Adventure* on an original Atari 2600 while being evaluated from camera pixels and standard joystick inputs only. The work is deliberately not an attempt to establish broad generality. Instead, it makes the systems boundary and evidence requirements explicit before results are collected.

1.1 Agent57 as motivation, not a target to reproduce

Agent57 is the most directly relevant reference point: it trained a family of policies ranging from exploratory to exploitative and adaptively prioritized them, reporting the first deep-RL result above the standard human benchmark on all 57 Atari games [1]. Its emphasis on weak-task performance

and the exploration–exploitation trade-off is a useful design lesson for our proposed memory and exploration components.

Our study is complementary rather than a like-for-like comparison. Agent57’s claim concerns a software Atari benchmark; its evaluation interface supplies an emulator’s observations, actions, resets, and learning signals. It does not test the additional physical-system questions introduced here: camera and display variation, externally applied controller contacts, end-to-end latency, and an audit trail that lets an observer see what was applied to the console. Those are not defects in Agent57—they were outside its stated benchmark scope—but they delimit what its reported score can establish about deployed physical play.

We therefore challenge three narrower assumptions rather than Agent57’s Atari57 result: (i) that high simulator performance is sufficient evidence of real-console transfer; (ii) that average or normalized score alone is an adequate report when hardware timing and failures matter; and (iii) that the large-scale, adaptive-policy approach is the right efficiency baseline for a single accessible physical setup. We will compare our method only with declared same-budget baselines, report all failures and compute, and never interpret a result on one game as evidence of general Atari competence.

2 Research questions and hypotheses

RQ1: transfer. Can an emulator-trained agent achieve a predefined *Adventure* objective on original hardware using only camera observations and joystick actions? The primary hypothesis is that latency- and appearance-randomized training improves zero-shot transfer over a baseline trained with a fixed, idealized emulator interface.

RQ2: efficiency. At a fixed interaction-frame budget, does the proposed memory/exploration design improve the selected task metric relative to a recurrent pixel-policy baseline? The study will report interaction frames, wall-clock training time, and compute/energy estimates rather than only final score.

RQ3: observability. Can every hardware episode be reconstructed from synchronized camera video, requested actions, applied relay actions, and configuration metadata? The hypothesis is operational: a third party can verify alignment and reproduce the reported outcome from the released log.

3 Experimental system

3.1 Environment and task

The initial task is Atari 2600 *Adventure*, selected because its standard joystick/fire interface combines navigation, object interaction, delayed outcomes, and room-to-room memory. Before collecting results, we will declare the ROM hash, game variation, reset/start procedure, console region, and a finite set of task endpoints. Candidate endpoints include reliable movement, carrying an object across a room transition, returning to a previously observed room, and task completion. The final endpoint definitions and scoring rules must be fixed before the evaluation runs are inspected.

[TODO: System figure: Atari → display → fixed camera → Linux box → Nano → relay contacts → Atari controller port. Label the synchronized log.]

Figure 1: Planned physical-console evaluation boundary.

3.2 Emulator training

The emulator API exposes `reset(rom, seed)` and `step(action, repeat_frames)`. During training, reward and terminal signals may be used for learning; the physical-console evaluation policy does not receive score, RAM, room identifiers, object coordinates, or emulator state. The baseline is a recurrent pixel policy with prioritized replay. The proposed agent augments it with a compact latent world model, uncertainty-aware exploration, and conditional planning. Model capacity, optimizer, replay policy, seeds, action repeat, and total interaction budget will be disclosed.

To prepare for transfer, training randomizes camera-relevant appearance and timing: random crops and perspective perturbations, colour and brightness changes, skipped frames, action holds, and measured camera/display/control delay. Randomization ranges will be chosen from calibration data, not tuned against final hardware scores.

3.3 Original-console interface

Figure 1 is a placeholder for the final system diagram. An original Atari 2600 drives its normal display. A fixed USB camera mounted on a two-axis Robotis head observes that display; pan and tilt are used only for setup and then fixed. An existing Linux box hosts camera capture, policy inference, preview, and logging. It transmits bounded commands over USB serial to a classic Arduino Nano. The Nano validates legal joystick masks and, through a ULN2803A driver and five normally-open relays, closes the standard up, down, left, right, and fire lines to controller common. The relay contacts are the sole electrical connection from the agent apparatus to the controller port.

The original joystick remains unmodified and is used separately for human reference runs. It is never connected in parallel with the relay bridge. Console power, reset, difficulty, and game selection remain manual in this experiment. A direct composite-to-USB capture alternative may be evaluated later but is not part of the primary configuration.

3.4 Safety and failure behaviour

All relay contacts must start open. The Nano opens every contact on boot, malformed commands, serial loss, or watchdog timeout; it rejects opposing directions and bounds a hold duration unless the host renews it. Hardware is commissioned with continuity tests before console connection. No controller port pin powers relay coils or receives a driven host voltage.

4 Evaluation protocol

4.1 Conditions

We will compare the following conditions using the same declared game configuration:

1. recurrent emulator baseline, evaluated in the emulator;
2. proposed agent, evaluated in the emulator;
3. frozen proposed agent, evaluated on original hardware;

4. where justified by a preregistered budget, proposed agent with limited, documented hardware fine-tuning; and
5. human reference runs using the original joystick.

Human runs are descriptive references, not training demonstrations and not a claim of statistical equivalence without a preregistered protocol.

4.2 Metrics

The primary metric will be the preregistered task endpoint success rate across held-out seeds/starts, with binomial confidence intervals. Secondary metrics are time to endpoint, score where lawfully observable and separately recorded, survival/progress measures, emulator interaction frames, wall-clock time, compute/energy, and median/90th-percentile/worst-case end-to-end latency. We will report all attempted hardware episodes, including aborts, watchdog trips, dropped-frame runs, and failures to start.

4.3 Observability and audit trail

Each episode produces camera video or timestamped frames, requested action, applied relay mask, monotonic timestamps, configuration hash, and outcome. A playback tool overlays actions and timing on recorded gameplay. We will test synchronization by issuing repeated visible actions and measuring the time from host command through relay application to the first responding camera frame. The released artifact must permit an independent reviewer to inspect behaviour without access to hidden console or emulator state.

5 Analysis plan

We will define the sample size, seeds/starts, exclusion rules, randomization ranges, and confidence-interval method before collecting final hardware results. Hyperparameter selection occurs on disjoint development seeds and may not use held-out hardware outcomes. We will publish learning curves and per-run results rather than only maxima, and distinguish zero-shot transfer from any hardware-adapted condition. Claims will be limited to the declared task, hardware configuration, and compute budget.

6 Limitations and responsible reporting

This single-game study cannot establish general intelligence, broad Atari competence, or human-level performance. A camera-plus-display path introduces latency and visual artefacts that are absent from emulator pixels; conversely, it is a useful test of external perception and action. Original-console variation, display properties, ROM rights, and human-reference methodology must be reported. We will release only material we are entitled to share and will avoid distributing copyrighted ROMs.

7 Reproducibility checklist

- ROM hash and lawful acquisition procedure (without redistributing ROMs).
- Console, display, camera, Linux host, Nano, relay and firmware revisions.

- Camera crop, exposure, gain, white balance, frame rate, and Robotis head positions.
- Relay wiring diagram, continuity tests, watchdog tests, and command protocol.
- Training code, environment version, seeds, budgets, checkpoints, and dependency lockfile.
- Raw/re-encoded gameplay logs, action/relay traces, latency calibration, analysis code, and complete result tables.

8 Conclusion

This draft specifies a modest and auditable experiment: an agent learns in an emulator, plays *Adventure* on an original Atari 2600 through normal human-style controls, and leaves an inspectable record of what it saw and did. Empirical conclusions are intentionally deferred until the specified evidence has been collected.

References

- [1] Adrià Puigdomènech Badia, Bilal Piot, Steven Kapturowski, et al. Agent57: Outperforming the atari human benchmark. *arXiv preprint arXiv:2003.13350*, 2020.
- [2] Marc G. Bellemare, Yavar Naddaf, Joel Veness, and Michael Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47:253–279, 2013.
- [3] Steven Kapturowski, Georg Ostrovski, Will Dabney, John Quan, Lennart Martens, and Rémi Munos. Recurrent experience replay in distributed reinforcement learning. In *International Conference on Learning Representations*, 2019.
- [4] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- [5] Abdelrhman Shaheen, Anas Badr, Ali Abohendy, Hatem Alsaadawy, Nadine Alsayad, and Ehab H. El-Shazly. Reinforcement learning in strategy-based and atari games: A review of google deepminds innovations. *arXiv preprint arXiv:2502.10303*, 2025.